Learning Agents: A Comprehensive Report on Definitions, Architectures, Algorithms, and Applications

Introduction

The evolution of artificial intelligence has moved from static prediction models to conversational chatbots and, most recently, to autonomous systems that can perceive, plan, act, and improve over time. Among these, the learning agent has emerged as a critical category because it closes the loop between interaction and improvement. According to industry forecasts cited by Baidu Qianfan, Gartner predicts that by 2026 more than 50% of enterprises will deploy AI agents to automate key business processes (Baidu Qianfan, 2025). This report examines the learning agent in depth: its definition, core architecture, learning algorithms, perception and decision mechanisms, representative application scenarios, frameworks, challenges, and future trajectories.

A learning agent is not simply a language model that answers questions. It is a system that combines a large language model (LLM) with tools, memory, planning, and feedback mechanisms to pursue goals and adapt its behavior based on experience (Kimi, 2026). This report synthesizes available sources to provide an objective, structured account of learning agents as they are understood and deployed in current practice.

Defining the Learning Agent

From Chatbots to Learning Agents

A common misunderstanding is to equate AI agents with chatbots such as ChatGPT. The difference can be summarized as "speaking" versus "doing." A chatbot primarily generates text in response to prompts, while an agent can decompose a complex goal, invoke external tools, execute actions, check results, and continue iterating until the task is completed (Baidu Qianfan, 2025). A learning agent adds another dimension: it updates its behavior based on feedback or past experience. When previous results are incomplete, inaccurate, or inefficient, a learning agent may adjust subsequent actions (Kimi, 2026).

Oracle defines an AI agent as a software entity that perceives its environment, takes actions, and learns from experience, with goals set by humans (Oracle, 2024). This learning ability distinguishes it from a purely reactive system because an LLM, once trained, is static; however, by observing which data, prompts, and tool combinations yield better outcomes, the agent can improve task performance over time (Oracle, 2024).

Core Characteristics of a Learning Agent

Drawing from multiple sources, a learning agent can be described by five core characteristics:

Characteristic	Description
Autonomy	Acts without step-by-step human instructions; the user gives a goal and the agent decides the next actions (Kimi, 2026).
Planning	Decomposes a vague goal into concrete subtasks and checkpoints (Kimi, 2026).
Tool use	Calls web search, file readers, code execution, APIs, browsers, and generation tools to perform real work (Kimi, 2026).
Memory	Maintains short-term context and long-term knowledge to keep coherence across a workflow (AWS, n.d.).
Learning and feedback	Evaluates outcomes, stores successful solutions, and corrects mistakes, thereby improving future performance (IBM, n.d.; Baidu Qianfan, 2025).

Learning agents are one of several agent types identified in the AI literature. They differ from simple reflex agents, model-based reflex agents, goal-based agents, and utility-based agents in that they explicitly incorporate feedback and experience to improve behavior (Kimi, 2026).

Core Architecture of a Learning Agent

The PEAS Framework

A classical way to understand agent architecture is through the PEAS model, which stands for Performance, Environment, Actuators, and Sensors (Baidu Qianfan, 2025). This model maps onto four functional blocks:

- **Perception** gathers information from text, images, sensors, databases, or multimodal inputs. For example, a customer-service agent may read both written messages and analyze emotional signals in voice (Baidu Qianfan, 2025).
- **Decision-making** acts as the agent's "brain," usually an LLM that interprets instructions, performs logical reasoning, and plans steps (Baidu Qianfan, 2025).
- **Action** executes decisions through tools such as search engines, APIs, payment interfaces, or hardware controls (Baidu Qianfan, 2025).
- **Reward/Learning** evaluates results and reinforces successful strategies while correcting unsuccessful ones (Baidu Qianfan, 2025).

This reward/learning block is what makes a learning agent genuinely adaptive.

LLM-Centric Architecture Components

In modern implementations, the learning agent's architecture can be abstracted into four core modules (AWS, n.d.):

1. **Reasoning engine**: Usually an LLM that understands intent, plans, and executes tasks. Its quality determines the agent's intelligence, and developers must manage prompt templates, reasoning chains, and inference costs (AWS, n.d.).
2. **Memory system**: Short-term memory maintains current session context, while long-term memory stores user preferences, historical interactions, and accumulated knowledge. Efficient retrieval and storage are key engineering challenges (AWS, n.d.).
3. **Orchestration module**: Coordinates components through task decomposition, workflow management, and tool-call sequencing. Frameworks such as LangGraph implement this via graph execution, while other systems use explicit planners (AWS, n.d.).
4. **Tool interface**: Standardizes access to APIs, databases, and external services. It must handle intelligent tool selection, retries, permission control, and security (AWS, n.d.).

Additionally, production learning agents require support services such as quality evaluation, identity and authorization, security and privacy protection, and observability (AWS, n.d.).

Architectural Principles for Production Systems

A set of 12 architectural principles for building production-grade agents emphasizes that prompt engineering alone is insufficient. Among the most relevant to learning behavior are: converting natural language into structured tool calls via JSON; treating prompts as code for maintainability; managing context explicitly as state; designing tools as structured outputs; and routing error information back into the context so that the agent can self-correct (UML, 2025). These principles support the learning loop by ensuring that failures are observable and can be incorporated into subsequent reasoning.

Learning Algorithms and Mechanisms

Feedback-Driven Improvement

One fundamental mechanism of learning agents is feedback-based iteration. IBM describes this as a feedback mechanism that improves the agent's reasoning and accuracy through iterative refinement; when the agent encounters obstacles, it can store solutions in a knowledge base to avoid repeating the same mistakes (IBM, n.d.). In the Baidu Qianfan framework, the reward/learning component evaluates whether a task was completed correctly and then strengthens or adjusts the decision logic accordingly (Baidu Qianfan, 2025).

Reinforcement Learning and Supervised/Unsupervised Updates

Oracle notes that reinforcement learning is a process in which an AI model refines its decision-making based on positive, neutral, and negative responses. An agent may also acquire new knowledge through additional supervised, unsupervised, or reinforcement learning training (Oracle, 2024). This means that a learning agent can improve not only by adjusting internal decision logic but also by receiving additional model training or fine-tuning based on deployment data.

Memory-Based Learning

Memory is central to learning. While LLMs are stateless by default, agents can provide persistent context that acts as long-term memory (Kimi, 2026). Oracle defines memory as the agent's ability to store information from past experience and retrieve it to make more informed decisions, which is essential for improving performance over time (Oracle, 2024). In education agents, for instance, the 以正教育 large model agent stores students' weak knowledge points and error-prone items in memory and, based on forgetting curves, initiates review and consolidation activities (The Paper, 2024).

Algorithmic Patterns for Learning Loops

Learning agents often follow structured reasoning and reflection cycles. IBM describes ReAct (Reasoning and Action) as a "think-act-observe" loop that enables step-by-step problem solving and iterative response improvement, while ReWOO (Reasoning Without Observation) offers an alternative that reduces repeated observation calls (IBM, n.d.). More advanced architectures include planning, reflection, chain-of-thought, and tree-of-thoughts patterns, which allow an agent to plan tasks, critique its own outputs, and branch into alternative reasoning paths when needed (Wayland Zhang, 2026).

Evaluation-Centric Learning in Practice

A concrete example of learning through evaluation appears in the AWS Bedrock ESG report agent. This system uses a "generate – evaluate – iterate" loop with LangGraph: each topic report is generated, assessed against professional standards and expert experience, and sent back for revision until it reaches a predefined quality threshold. The system's evaluation score must exceed a set threshold before the report is accepted; otherwise, feedback triggers regeneration (AWS Bedrock, n.d.). This demonstrates a practical learning mechanism where the agent's output is explicitly scored and improved through iterative refinement.

Perception and Decision-Making Mechanisms

Perception

A learning agent must sense its environment before it can act. Perception can include extracting requirements from text, recognizing objects from images, reading sensor data from smart-home devices, and integrating multiple sources simultaneously. For example, a customer-complaint agent may read both written messages and analyze emotional fluctuations in voice to understand user intent fully (Baidu Qianfan, 2025). Oracle similarly notes that agents can use sensors, cameras, microphones, and other mechanisms to collect environmental information (Oracle, 2024). Retrieval-augmented generation (RAG) further enables agents to query external databases, enterprise systems, and documents to enrich their perception (Oracle, 2024).

Decision-Making and Planning

The agent's decision process begins with goal initialization and planning. Although the decision flow is autonomous, goals and rules are typically set by humans, and the agent then decomposes the objective into a plan with specific tasks and subtasks (IBM, n.d.). The LLM acts as the core

reasoning engine, interpreting vague requests and generating structured outputs. For example, natural language is converted into a JSON action object that calls a system API, bridging natural language control and programmatic execution (UML, 2025).

A well-designed agent also incorporates checkpoints so that it can recognize when more information is needed, when source quality is insufficient, or when human review is required (Kimi, 2026). The typical workflow is: understand the user's goal, make a task plan, collect information, use tools, take action, check results, and continue iterating if necessary (Kimi, 2026).

Action and Tool Use

Action is the tangible output of the agent. Tools include web search for real-time information, file readers for PDFs and spreadsheets, code execution for analysis, APIs for external data, and browser or application connectors for workflow operations (Kimi, 2026). The action phase may also involve communication tasks such as composing emails, transcribing audio, or asking clarifying questions (Oracle, 2024). Structured tool calls are preferred because they separate model reasoning from business logic and allow better error handling and orchestration (UML, 2025).

Application Scenarios for Learning Agents

Education and Intelligent Tutoring

Education is one of the most visible domains for learning agents. A survey of AI Agent applications in education identifies roles such as intelligent teachers, tutors, evaluators, and administrative assistants (The Paper, 2024). Specific examples include:

- **Khanmigo** is a GPT-4-based agent developed by Khan Academy. It acts as a virtual tutor, providing personalized guidance, teacher assistance, and writing coaching. In mathematics, it does not simply give answers; it predicts likely student errors and guides reasoning step by step (The Paper, 2024).
- **Jagoda.ai** is an AI-powered online tutoring agent covering more than 30 subjects and 20 languages. It adapts to the student's learning style and school curriculum, provides step-by-step solutions, and supports follow-up questions to deepen understanding (The Paper, 2024).
- **GPT Researcher** is an autonomous research agent that generates detailed, factual, and unbiased reports through parallel agents, improving speed and reliability over synchronous operation (The Paper, 2024).
- 以正教育大模型**Agent** deploys three types of agents—lesson plan agents, assistant teaching agents, and study companion agents—that interact autonomously. The assistant teaching

agent stores student performance data and weaknesses in memory, while the study companion agent uses that memory to deliver targeted review and exercises according to forgetting cycles（The Paper, 2024）.

Enterprise Workflow and Business Automation

Learning agents are also being adopted in enterprise environments. IBM describes a multi-agent legal research assistant built with Dynamiq and IBM watsonx Orchestrate for a large insurance client. The system routes queries through a low-cost classifier and escalates complex cases to a more powerful research agent, reducing contract review time from 90 minutes to 45 minutes while maintaining auditability and cost-effectiveness (IBM, n.d.). The 2024 China AI Agent Best Practice Cases list includes deployments in government, audio creation, education marketing, and telecommunications, illustrating the breadth of real-world adoption (Shaqiu, 2024）.

Research, Report Generation, and Coding

Learning agents excel at deep research and content generation tasks. Kimi's agent mode, for instance, supports advanced multi-source web search, long-context processing, multimodal reasoning, and multi-round iterative research, allowing users to refine queries and receive results in multiple formats (Kimi, 2026). The AWS Bedrock ESG report agent uses LangGraph's Map-Reduce architecture, semantic search over standards and historical reports, and a generation-evaluation loop to produce high-quality ESG reports that meet professional standards (AWS Bedrock, n.d.）.

Representative Frameworks and Platforms

Several platforms and frameworks support the development of learning agents. A concise comparison is shown below.

Platform / Framework	Primary Focus	Key Features	Source
Kimi Agent Mode	Deep research and multimodal task execution	Long context window, multi-source search, multi-round iterative output	（Kimi, 2026）
LangGraph	Workflow orchestration	Map-Reduce, cyclic graphs, tool calls, evaluation loops	（AWS Bedrock, n.d.）

Platform / Framework	Primary Focus	Key Features	Source
Shannon / Kocoro	Enterprise multi-agent orchestration	Planning, reflection, memory, token budgets, sandboxing, multi-agent patterns	(Wayland Zhang, 2026)
OCI Generative AI Agents	Enterprise RAG and agent deployment	Combines LLM with RAG, personalized agents embedded in business applications	(Oracle, 2024)
LearnAgent	Developer knowledge base and protocol tracking	Tracks MCP, A2A interactions, tool-call patterns, and memory mechanisms	(LearnAgent, n.d.)

In addition, production systems increasingly require AgentOps practices that bring observability, security, and evaluation into the agent lifecycle. AWS emphasizes that building a "production-ready" agent requires not only model and tool integration but also quality assessment, identity and authorization, privacy protection, and monitoring (AWS, n.d.). The 12 principles for agent architecture similarly stress the need for small, focused agents, lifecycle APIs that support pause/resume, and stateless mergers for horizontal scalability in enterprise deployments (UML, 2025).

Challenges and Limitations

Despite their promise, learning agents face significant challenges.

First, learning capability is often constrained by privacy and security concerns. In user-facing products, unrestricted learning may be limited because it could expose sensitive data or enable unwanted personalization (Kimi, 2026). AWS highlights threats from the OWASP Agentic AI threat model, including memory poisoning, tool misuse, permission abuse, and identity deception, which require layered defenses at input, reasoning, tool-call, and output stages (AWS, n.d.).

Second, autonomous agents can make mistakes that compound across long workflows. The 2024 China AI Agent Best Practice report recommends human-in-the-loop operation for complex or ambiguous situations, with clear process guardrails and careful management of autonomy expectations (Shaqiu, 2024). Similarly, Oracle notes that agents should be given autonomy gradually, just like new employees, and that clear, measurable goals are essential for success (Oracle, 2024).

Third, multi-agent coordination introduces additional complexity. Without clear task boundaries, shared context, and review mechanisms, parallel agents may duplicate work or produce inconsistent results (Kimi, 2026). Governance mechanisms such as token budgets, access controls, and sandbox isolation become necessary at enterprise scale (Wayland Zhang, 2026).

Finally, evaluation remains difficult. While LLM-as-a-judge combined with human audit is a common approach, robust quality assurance for agent reasoning and task completion is still an engineering challenge (AWS, n.d.).

Future Trends

Several trends are shaping the future of learning agents.

From single agents to multi-agent systems. Large tasks are increasingly decomposed across specialized subagents that search, analyze, write, and review in parallel, under a supervisor or handoff pattern (Kimi, 2026; Wayland Zhang, 2026). This requires better coordination, shared context, and quality control, but it can dramatically improve throughput and specialization.

Productionization through AgentOps. As agents move from prototypes to production, practices such as observability, sandboxing, token budget control, and lifecycle management are becoming standard. Frameworks like Shannon explicitly build in these enterprise-grade capabilities (Wayland Zhang, 2026), while AWS describes the extension from DevOps to AgentOps as necessary to manage the nondeterminism of LLMs (AWS, n.d.).

Protocol standardization. The emergence of MCP (Model Context Protocol) and A2A (Agent-to-Agent) protocols is being tracked closely by developer communities such as LearnAgent, indicating movement toward interoperable tool and agent communication standards (LearnAgent, n.d.).

Background and always-on agents. Research and practitioner sources point to background agents, computer-use agents, and agentic coding as frontier practices. These agents operate continuously, handling tasks such as deep research, code generation, and browser automation with limited human intervention (Wayland Zhang, 2026).

Enterprise adoption and economic impact. The forecast that more than 50% of enterprises will deploy AI agents by 2026 signals a shift from experimental chatbots to business-critical automation (Baidu Qianfan, 2025). As learning agents become better at using memory, feedback, and tools, they are likely to evolve from task-specific assistants into generalized digital workers capable of managing end-to-end business processes.

Conclusion

The learning agent represents a meaningful step beyond static language models and scripted chatbots. By combining LLM reasoning with perception, planning, tool use, memory, and feedback-driven adaptation, learning agents can pursue complex goals and improve their performance through experience. Their architectures incorporate reasoning engines, memory systems, orchestration modules, and tool interfaces, while their learning mechanisms range from reinforcement learning and iterative refinement to memory-based knowledge storage and evaluation loops. Applications in education, enterprise automation, research, and report generation demonstrate early real-world value, with measurable outcomes such as reduced legal review time from 90 to 45 minutes (IBM, n.d.). However, significant challenges remain in privacy, security, multi-agent coordination, and evaluation. Looking ahead, the integration of AgentOps, protocol standards, and multi-agent orchestration is likely to push learning agents from narrowly scoped tools toward robust, enterprise-grade autonomous systems.

References

AWS. (n.d.). *Agentic AI* 基础设施实践系列一：*Agent* 应用开发. 亚马逊AWS官方博客. https://aws.amazon.com/cn/blogs/china/agentive-ai-infrastructure-practice-series-1

AWS. (n.d.). 利用*AWS Bedrock*构建智能报告生成*Agent*. 亚马逊AWS官方博客. https://aws.amazon.com/cn/blogs/china/building-intelligent-report-generation-agent-with-aws-bedrock

Baidu Qianfan. (2025, September 10). *AI*智能体的定义、工作原理、类型及应用场景、面临的挑战与未来展望. 百度智能云千帆社区. https://qianfan.cloud.baidu.com/qianfandev/topic/686456

IBM. (n.d.). 什么是 *AI* 智能体？ IBM. https://www.ibm.com/cn-zh/think/topics/ai-agents

Kimi. (2026, September 4). *AI*智能体详解：工作原理、类型与示例. Kimi Resources. https://www.kimi.com/resources/ai-agent

Kimi. (2026, August 12). 现实生活中你应该了解的 10 个 *AI Agent* 示例. Kimi Resources. https://www.kimi.ai/zh-hans/resources/ai-agents-examples

LearnAgent. (n.d.). *AI Agent* 学习与开发中文知识库. LearnAgent. https://learnagent.org

Oracle. (2024, December 12). 什么是 *AI Agent*？ Oracle 中国. https://www.oracle.com/cn/artificial-intelligence/ai-agents

Shaqiu. (2024). 2024中国*AI Agent*最佳实践案例*TOP20*重磅发布. 沙丘社区. https://www.shaqiu.cn/article/4NDrLga8Ymx1

The Paper. (2024, September 18). 从国内外10个智能体案例，看*AI Agent*在教育领域的应用. 澎湃新闻·湃客. https://www.thepaper.cn/newsDetail_forward_28772976

UML. (2025, October 17). *Agent*架构设计：12大原则详解，从0到1构建可落地的*Agent*系统. UML. http://www.uml.org.cn/ai/202510171.asp

Wayland Zhang. (2026). *AI Agent* 架构：从单体到企业级多智能体. Wayland Zhang. https://waylandz.com/ai-agent-book